TCP Congestion Control

Data Network — Transport Layer

2026-04-16

Table of contents

10 **16**

1 (Overview)

TCP Congestion Control (congestion) , **- - (end-to-end)** . IP , TCP **(loss event)** (implicit) .

> **Note**
>
> **Flow Control vs. Congestion Control**
>
> | Flow Control | Congestion Control |
> |---|---|
> | `rwnd` (Receive Window) Sender Receiver | (Timeout / 3 Dup ACK) |

1.1

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\underbrace{cwnd}_{\text{congestion window}}, \underbrace{rwnd}_{\text{flow control window}})$$

- **`cwnd` (Congestion Window)**: unACKed . .
- **`ssthresh` (Slow Start Threshold)**: Slow Start Congestion Avoidance .
- :

$$\text{rate} \approx \frac{cwnd}{RTT} \quad [\text{bytes/sec}]$$

---

2 (Costs of Congestion)

TCP , .

2.1 Scenario 1: (,)

, (λ_{in}) $C/2$ (delay) .

$$\lim_{\lambda_{in} \to C/2} \text{delay} = \infty$$

2.2 Scenario 2: + ( )

- → →
- **goodput < throughput**

$$\lambda_{out} < \lambda'_{in} \quad (\lambda'_{in} = \text{original} + \text{retransmitted})$$

2.3 Scenario 3: ( )

, .

> **Warning**
>
> : 1. goodput
> 2. **goodput**
> 3.

---

3 TCP

TCP (state) .

3.1 Slow Start (SS)

`cwnd` **(exponential)** .

:

- : $cwnd = 1$ MSS
- ACK : $cwnd \mathrel{+}= 1$ MSS
- : RTT `cwnd` **2**

$$cwnd(t + 1 \text{ RTT}) = 2 \cdot cwnd(t)$$

:

`cwnd >= ssthresh` Timeout ( )	Congestion Avoidance `ssthresh = cwnd/2`, `cwnd = 1 MSS`, SS
3 Duplicate ACKs	`ssthresh = cwnd/2`, `cwnd = ssthresh + 3`, Fast Recovery

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

def simulate_tcp(initial_ssthresh=16, max_rounds=20, loss_at=None):
    cwnd = 1
    ssthresh = initial_ssthresh
    cwnd_history = [cwnd]
    ssthresh_history = [ssthresh]
    states = ['SS']

    for rtt in range(1, max_rounds):
        if loss_at and rtt in loss_at:
            ssthresh = max(cwnd // 2, 1)
            loss_type = loss_at[rtt]
            if loss_type == 'timeout':
                cwnd = 1
                states.append('SS')
            else:  # 3 dup ACK
                cwnd = ssthresh + 3
                states.append('FR')
        else:
            if cwnd < ssthresh:  # Slow Start
                cwnd *= 2
                states.append('SS')
            else:  # Congestion Avoidance
                cwnd += 1
                states.append('CA')

        cwnd_history.append(cwnd)
        ssthresh_history.append(ssthresh)

    return cwnd_history, ssthresh_history, states

cwnd_h, ssthresh_h, states = simulate_tcp(
    initial_ssthresh=16,
    max_rounds=25,
    loss_at={8: '3dup', 17: 'timeout'}
)

fig, ax = plt.subplots(figsize=(12, 6))

colors = {'SS': '#2196F3', 'CA': '#4CAF50', 'FR': '#FF9800'}
state_colors = [colors.get(s, '#9E9E9E') for s in states]

for i in range(len(cwnd_h)-1):
    ax.plot([i, i+1], [cwnd_h[i], cwnd_h[i+1]],
```

```
            color=state_colors[i+1], linewidth=2.5)
    ax.scatter(i, cwnd_h[i], color=state_colors[i], s=80, zorder=5)
ax.scatter(len(cwnd_h)-1, cwnd_h[-1], color=state_colors[-1], s=80, zorder=5)

ax.step(range(len(ssthresh_h)), ssthresh_h, where='post',
        linestyle='--', color='red', alpha=0.7, linewidth=1.5, label='ssthresh')

ax.set_xlabel('Transmission Round (RTT)', fontsize=12)
ax.set_ylabel('Congestion Window (MSS)', fontsize=12)
ax.set_title('TCP Congestion Control: cwnd Evolution\n(Loss at RTT=8: 3 Dup ACK, RTT=17: Timeou
ax.grid(True, alpha=0.3)

patches = [
    mpatches.Patch(color='#2196F3', label='Slow Start (SS)'),
    mpatches.Patch(color='#4CAF50', label='Congestion Avoidance (CA)'),
    mpatches.Patch(color='#FF9800', label='Fast Recovery (FR)'),
    mpatches.Patch(color='red', linestyle='--', label='ssthresh')
]
ax.legend(handles=patches, loc='upper right')
plt.tight_layout()
plt.show()
```

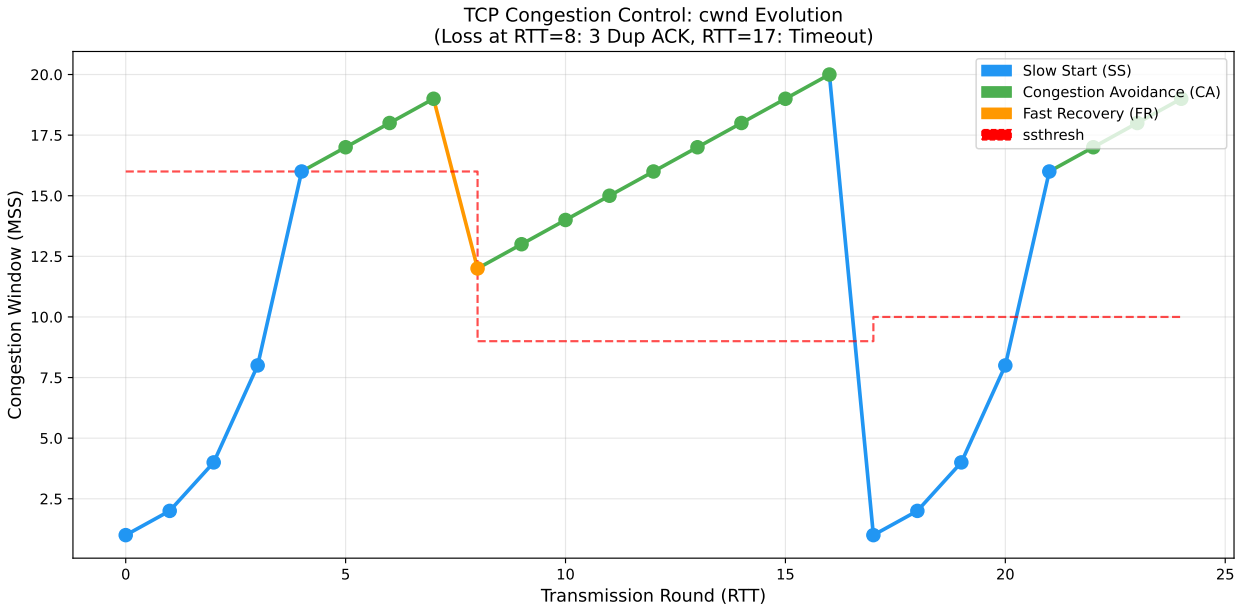


Figure 1: Slow Start: cwnd

3.2 Congestion Avoidance (CA)

`cwnd >= ssthresh` , `cwnd` **(linear)** .

:

- ACK : $cwnd \mathrel{+}= \frac{MSS^2}{cwnd}$
- : RTT $cwnd$ **+1 MSS** (Additive Increase)
- : $ssthresh = cwnd/2$, $cwnd = cwnd/2$ (Multiplicative Decrease)

AIMD (Additive Increase, Multiplicative Decrease) .

$$\text{AIMD:} \quad \underbrace{cwnd \mathrel{+}= 1 \text{ MSS per RTT}}_{\text{Additive Increase}}, \quad \underbrace{cwnd \leftarrow cwnd/2 \text{ on loss}}_{\text{Multiplicative Decrease}}$$

Tip

AIMD

, AIMD throughput **45°** **(equal share line)** .

- Additive Increase: throughput 1 →
- Multiplicative Decrease: →

Fair & Efficient .

3.3 Fast Retransmit & Fast Recovery (FR)

Fast Retransmit: 3 Duplicate ACK , . Dup ACK " " , timeout .

Fast Recovery (TCP Reno): Fast Retransmit Slow Start , `ssthresh` Congestion Avoidance .

```
# Fast Recovery     : 3 Dup ACKs
ssthresh = cwnd / 2
cwnd     = ssthresh + 3   # 3      Dup ACK
→ Fast Recovery
  -   Dup ACK   : cwnd += 1 MSS
  -   ACK     : cwnd = ssthresh, CA
  - Timeout     : cwnd = 1 MSS, SS
```

---

4 TCP Tahoe vs. TCP Reno

	TCP Tahoe	TCP Reno
Timeout	`cwnd=1`, SS	`cwnd=1`, SS
3 Dup ACK	`cwnd=1`, SS 3 Dup ACK Timeout	`cwnd=ssthresh+3`, Fast Recovery 3 Dup ACK

> **Note**
>
> **TCP Reno Tahoe**
> 3 Dup ACK , `cwnd` 1 MSS . Reno Fast Recovery throughput
> .

```
import matplotlib.pyplot as plt

#
# Tahoe  Reno   ssthresh=8   , 12 MSS  3 Dup ACK
# (Kurose   Figure 3.52  )

rounds_tahoe = list(range(0, 20))
cwnd_tahoe   = [1, 2, 4, 8, 9, 10, 11, 12, 1, 2, 4, 6, 8, 9, 10, 11, 12, 13, 14, 15]

rounds_reno  = list(range(0, 20))
cwnd_reno    = [1, 2, 4, 8, 9, 10, 11, 12, 7, 8,  9, 10, 11, 12, 13, 14, 15, 16, 17, 18]

ssthresh_series = [8]*8 + [6]*12  #     ssthresh=6

fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(rounds_tahoe, cwnd_tahoe, 'b-o', markersize=6, linewidth=2, label='TCP Tahoe')
ax.plot(rounds_reno,  cwnd_reno,  'g-s', markersize=6, linewidth=2, label='TCP Reno')
ax.step(range(20), ssthresh_series, 'r--', where='post', linewidth=1.5, alpha=0.8, label='ssth

ax.axvline(x=8, color='orange', linestyle=':', linewidth=2, label='3 Dup ACK @ round 8')
ax.set_xlabel('Transmission Round', fontsize=12)
ax.set_ylabel('cwnd (MSS)', fontsize=12)
ax.set_title('TCP Tahoe vs. Reno: cwnd Evolution\n(3 Duplicate ACKs at Round 8, ssthresh=8)',
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
ax.set_ylim(0, 20)

#
ax.annotate('Tahoe: cwnd→1\n(SS restart)',
            xy=(8, 1), xytext=(10, 3),
            arrowprops=dict(arrowstyle='->', color='blue'),
            fontsize=9, color='blue')
ax.annotate('Reno: cwnd→ssthresh+3\n(Fast Recovery)',
            xy=(8, 7), xytext=(10, 9.5),
            arrowprops=dict(arrowstyle='->', color='green'),
            fontsize=9, color='green')
plt.tight_layout()
plt.show()
```

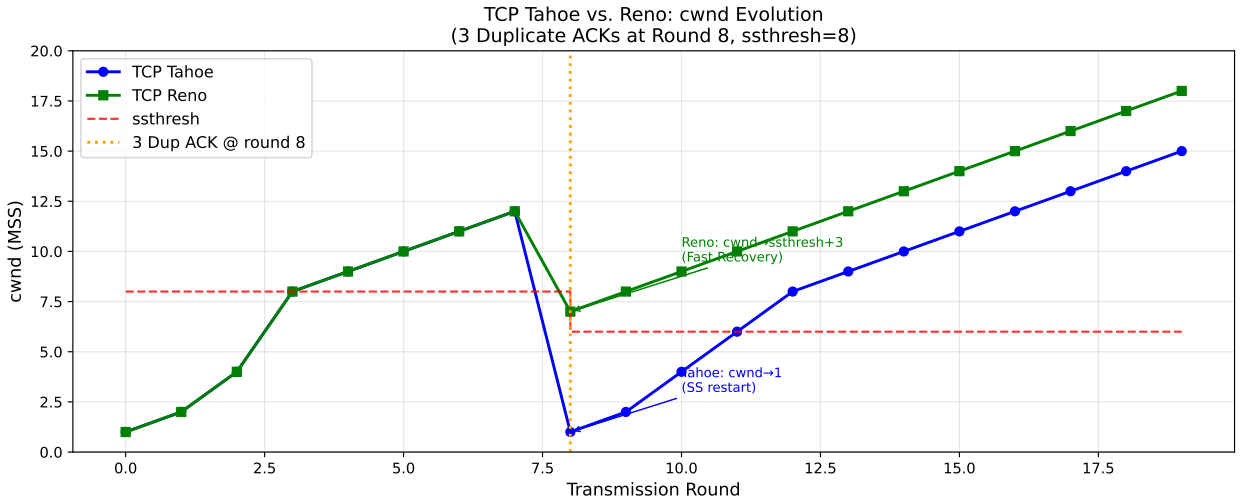


Figure 2: TCP Tahoe vs. Reno: 3 Dup ACK cwnd (ssthresh=8)

---

5 TCP FSM

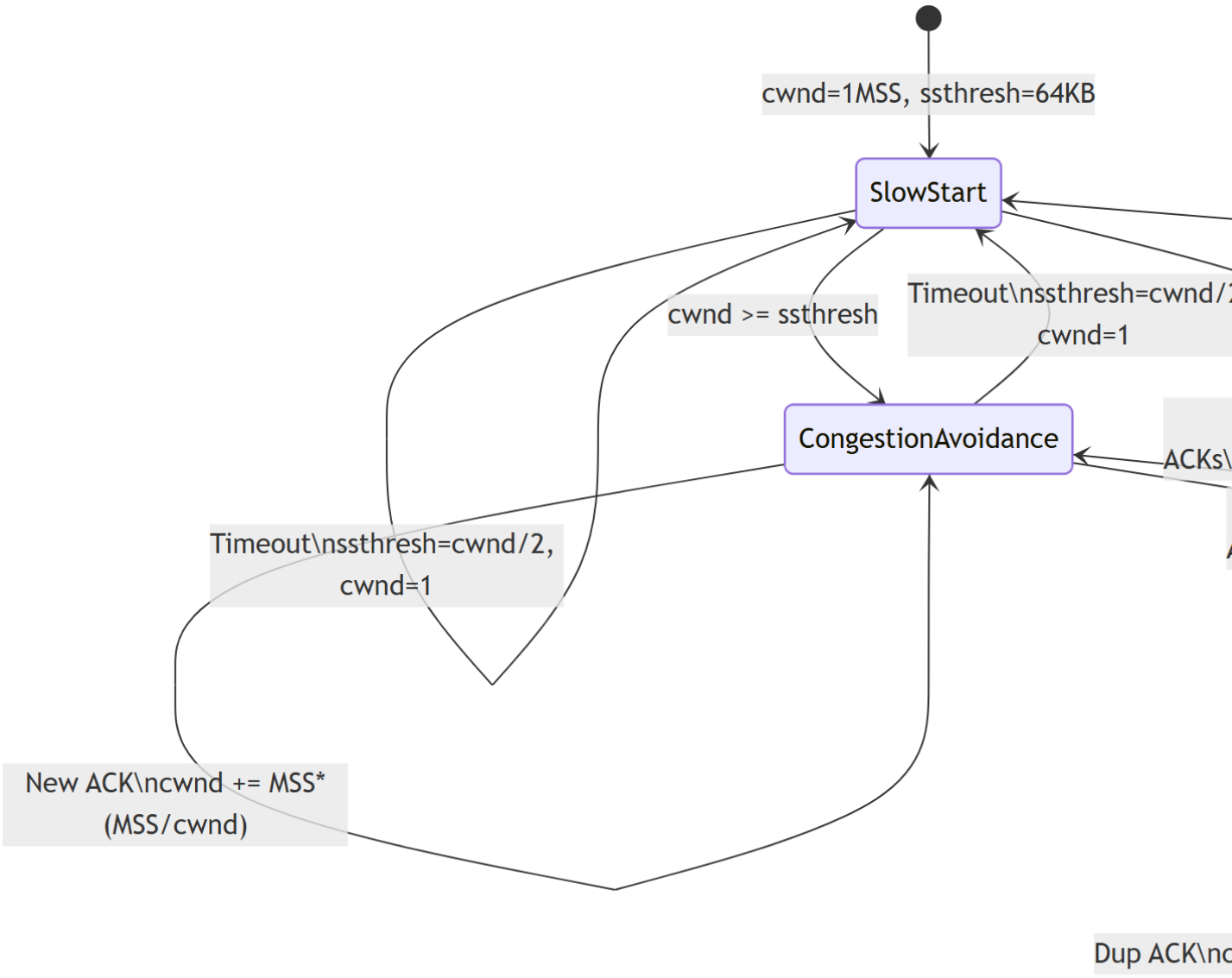
TCP Reno Finite State Machine .

```
stateDiagram-v2
    [*] --> SlowStart : cwnd=1MSS, ssthresh=64KB

    SlowStart --> SlowStart : New ACK\ncwnd += MSS
    SlowStart --> CongestionAvoidance : cwnd >= ssthresh
    SlowStart --> SlowStart : Timeout\nssthresh=cwnd/2, cwnd=1
    SlowStart --> FastRecovery : 3 Dup ACKs\nssthresh=cwnd/2\ncwnd=ssthresh+3

    CongestionAvoidance --> CongestionAvoidance : New ACK\ncwnd += MSS*(MSS/cwnd)
    CongestionAvoidance --> SlowStart : Timeout\nssthresh=cwnd/2, cwnd=1
    CongestionAvoidance --> FastRecovery : 3 Dup ACKs\nssthresh=cwnd/2\ncwnd=ssthresh+3

    FastRecovery --> FastRecovery : Dup ACK\ncwnd += MSS
    FastRecovery --> CongestionAvoidance : New ACK\ncwnd=ssthresh
    FastRecovery --> SlowStart : Timeout\nssthresh=cwnd/2, cwnd=1
```



---

6 TCP Throughput

6.1 (AIMD)

AIMD `cwnd` $W/2$ W . `cwnd = W` :

$$\overline{\text{throughput}} = \frac{3}{4} \cdot \frac{W}{RTT} \quad \text{[bytes/sec]}$$

: $\frac{W/2+W}{2} = \frac{3W}{4}$

6.2 Mathis Formula ( )

L [Mathis 1997]:

$$\overline{\text{throughput}} = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$$

> Tip
>
> ( )
> AIMD : - Additive Increase: $W/2$ W $W/2$ RTT - : $\sum_{k=W/2}^{W} k \cdot$
> $MSS \approx \frac{3W^2}{8} \cdot MSS$ - 1 $\rightarrow L = \frac{1}{\quad} = \frac{8}{3W^2}$ - $W = \sqrt{\frac{8}{3L}}$ throughput $\frac{1.22 \cdot MSS}{RTT\sqrt{L}}$

6.3 Long Fat Pipe

(: 10 Gbps, RTT=100ms) :

$$L \leq 2 \times 10^{-10}$$

. CUBIC, BBR TCP .

```
import numpy as np
import matplotlib.pyplot as plt

MSS = 1500  # bytes
RTT = 0.1   # seconds

L_range = np.logspace(-8, -2, 500)
throughput_Mbps = (1.22 * MSS) / (RTT * np.sqrt(L_range)) / 1e6

fig, ax = plt.subplots(figsize=(10, 5))
ax.loglog(L_range, throughput_Mbps, 'b-', linewidth=2.5)

#
targets = [(1e-6, '1Mbps'), (1e-4, ''), (1e-2, '')]
thresholds = {
    10000: ('10 Gbps', 'red'),
    1000:  ('1 Gbps', 'orange'),
    100:   ('100 Mbps', 'green'),
    10:    ('10 Mbps', 'blue'),
}
for tp, (label, color) in thresholds.items():
    L_needed = (1.22 * MSS / (RTT * tp * 1e6))**2
    ax.axhline(y=tp, color=color, linestyle='--', alpha=0.5, linewidth=1)
    ax.axvline(x=L_needed, color=color, linestyle=':', alpha=0.5, linewidth=1)
```

```
    ax.text(L_needed * 1.1, tp * 1.3, f'{label}\nL={L_needed:.1e}',
            fontsize=8, color=color)

ax.set_xlabel('Packet Loss Rate (L)', fontsize=12)
ax.set_ylabel('Throughput (Mbps)', fontsize=12)
ax.set_title('TCP Throughput vs. Loss Rate\n(Mathis Formula, MSS=1500B, RTT=100ms)', fontsize=
ax.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.show()
```

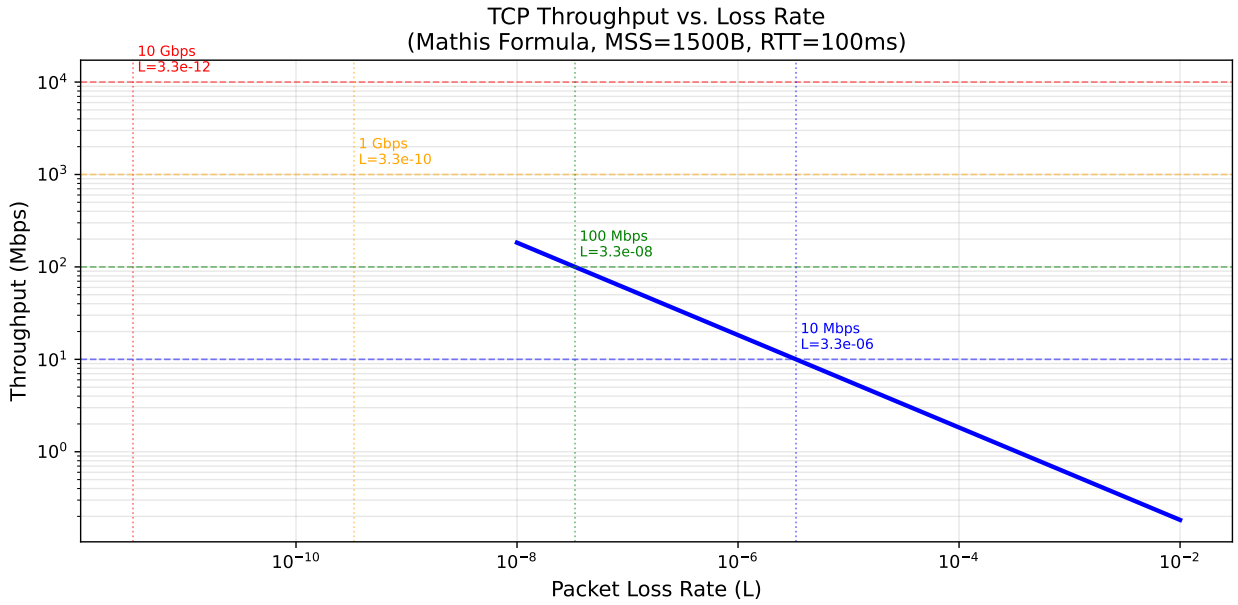


Figure 3: Mathis Formula: TCP Throughput (MSS=1500B, RTT=100ms)

---

7 TCP (Fairness)

7.1 AIMD

K TCP R , :

$$: \quad = \frac{R}{K}$$

:

- **AI** : $(+1, +1)$ — $(45°)$
- **MD** : $(0.5 \cdot x_1, 0.5 \cdot x_2)$ —

throughput ( ) ( ) .

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyArrowPatch

R = 10  #

fig, ax = plt.subplots(figsize=(8, 8))

#
ax.fill_between([0, R], [R, 0], [R, R], alpha=0.05, color='red', label='Over-capacity')
ax.fill_between([0, R], [0, 0], [R, 0], alpha=0.05, color='green', label='Under-capacity')

#
ax.plot([0, R], [0, R], 'k--', linewidth=1.5, label='Equal share (45°)')
ax.plot([0, R], [R, 0], 'r-', linewidth=2, alpha=0.7, label='Capacity limit (x +x =R)')

# AIMD
np.random.seed(42)
x1, x2 = 1.0, 7.0  #    (    )
trajectory = [(x1, x2)]

for _ in range(20):
    # AI:   RTT  +1
    x1 += 1; x2 += 1
    #       MD
    if x1 + x2 > R:
        x1 *= 0.5; x2 *= 0.5
    trajectory.append((x1, x2))

xs, ys = zip(*trajectory)
ax.plot(xs, ys, 'b-o', markersize=5, linewidth=1.5, alpha=0.8, label='AIMD trajectory')
ax.scatter(xs[0], ys[0], s=150, c='blue', marker='^', zorder=6, label='Start')
ax.scatter(xs[-1], ys[-1], s=150, c='red', marker='*', zorder=6, label='Converged point')

#
mid_idx = 5
ax.annotate('', xy=(xs[mid_idx]+1, ys[mid_idx]+1),
            xytext=(xs[mid_idx], ys[mid_idx]),
            arrowprops=dict(arrowstyle='->', color='green', lw=2))
ax.text(xs[mid_idx]+0.5, ys[mid_idx]+0.3, 'AI (+1,+1)', fontsize=9, color='green')

ax.annotate('', xy=(xs[mid_idx+1]*0.5, ys[mid_idx+1]*0.5),
            xytext=(xs[mid_idx+1], ys[mid_idx+1]),
            arrowprops=dict(arrowstyle='->', color='red', lw=2))
ax.text(xs[mid_idx+1]*0.5+0.2, ys[mid_idx+1]*0.5+0.5, 'MD (×0.5)', fontsize=9, color='red')
```

```
ax.set_xlim(0, R+1)
ax.set_ylim(0, R+1)
ax.set_xlabel('Throughput of Connection 1 (Mbps)', fontsize=12)
ax.set_ylabel('Throughput of Connection 2 (Mbps)', fontsize=12)
ax.set_title('AIMD Fairness Convergence\n(Two TCP Connections, Link Capacity R=10)', fontsize=
ax.legend(fontsize=10, loc='upper right')
ax.grid(True, alpha=0.3)
ax.set_aspect('equal')
plt.tight_layout()
plt.show()
```

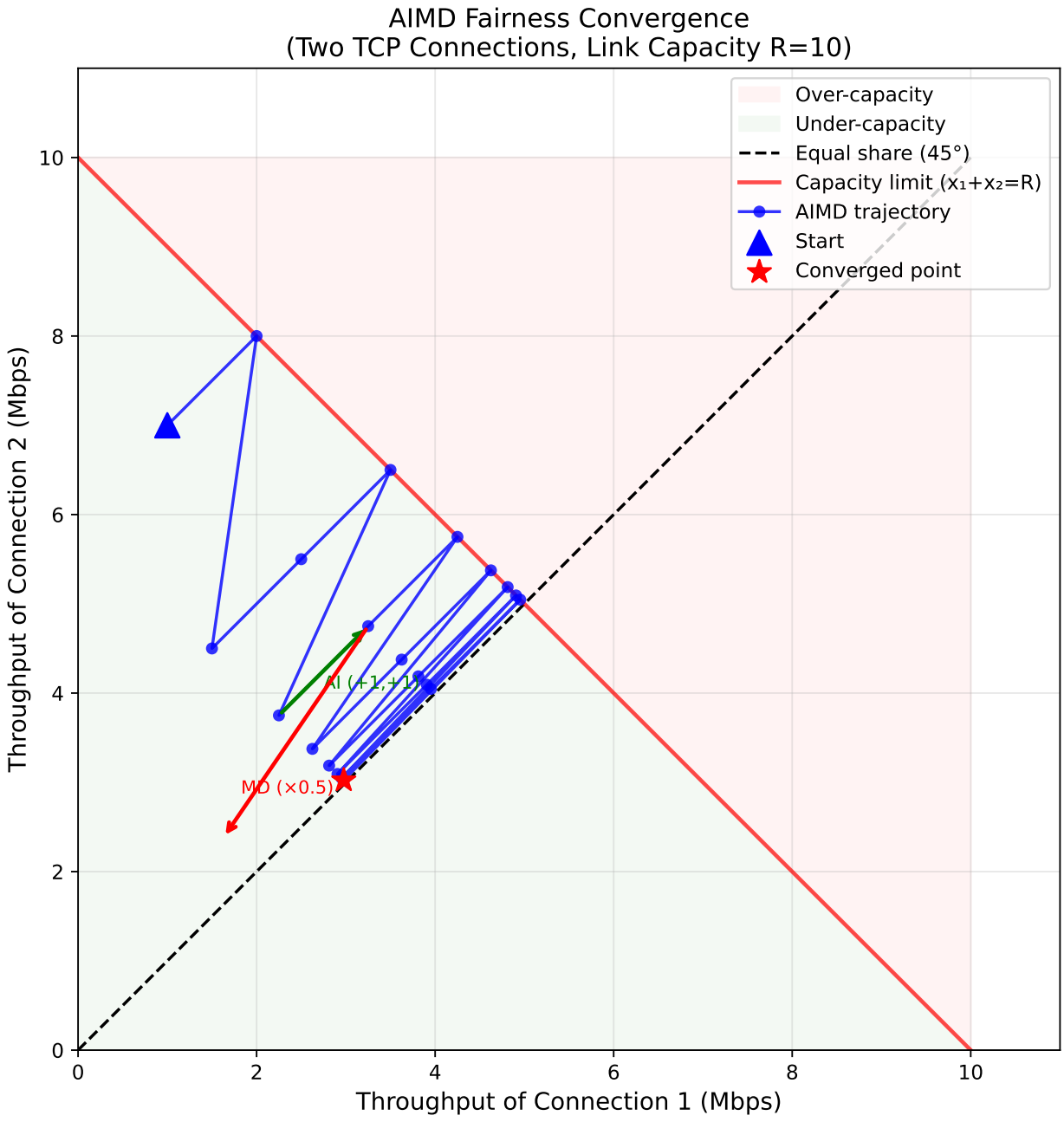


Figure 4: AIMD Fairness: TCP Throughput

7.2

- **RTT** : RTT `cwnd` → **RTT** **throughput**
- **TCP** : N N
- **UDP** : UDP TCP

8 Explicit Congestion Notification (ECN)

TCP **(loss)** , ECN (network-assisted) .

8.1

```
[ ]                → [ ]
          IP datagram: ECN=10 (capable)
                   ↓ (       )
          IP datagram: ECN=11 (CE: Congestion Experienced)
[ ] ←                 [ ]
          TCP ACK: ECE=1 (Echo CE  )
[ ]: cwnd      , CWR=1
```

ECN	
00	ECN
10 or 01	ECN (ECT: ECN-Capable Transport)
11	(CE: Congestion Experienced)

> **Note**
>
> **ECN**
> , . (DCTCP) .

9

9.1

throughput	rate $\approx cwnd/RTT$
Mathis throughput	$\overline{T} = \frac{3}{4} \cdot \frac{W}{RTT}$ $\overline{T} = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$ $= R/K$

9.2 cwnd

		cwnd
Slow Start	New ACK	`cwnd += 1 MSS`
Slow Start	Timeout	`ssthresh=cwnd/2`, `cwnd=1`
Slow Start	3 Dup ACK	`ssthresh=cwnd/2`, `cwnd=ssthresh+3` → FR
Congestion Avoidance	New ACK	`cwnd += MSS*(MSS/cwnd) +1 MSS/RTT`
Congestion Avoidance	Timeout	`ssthresh=cwnd/2`, `cwnd=1`
Congestion Avoidance	3 Dup ACK	`ssthresh=cwnd/2`, `cwnd=ssthresh+3` → FR
Fast Recovery	Dup ACK	`cwnd += 1 MSS`
Fast Recovery	New ACK	`cwnd=ssthresh` → CA
Fast Recovery	Timeout	`ssthresh=cwnd/2`, `cwnd=1` → SS

9.3

Q. `ssthresh = 8 MSS`, `cwnd = 1 MSS` TCP Reno .
Round 12 3 Dup ACK , Round 12 `cwnd` 12 MSS :

1. `ssthresh` ?
2. Fast Recovery `cwnd` ?
3. ACK `cwnd` ?

A.

1. $ssthresh = 12/2 = 6$ MSS
2. $cwnd = 6 + 3 = 9$ MSS

3. $cwnd = ssthresh = 6$ MSS, Congestion Avoidance

10

- Kurose, J. F. & Ross, K. W. (2016). *Computer Networking: A Top-Down Approach*, 7th Edition. Pearson.
  - Chapter 3.6: Principles of Congestion Control
  - Chapter 3.7: TCP Congestion Control
- Jacobson, V. (1988). Congestion Avoidance and Control. *ACM SIGCOMM*.

- Mathis, M. et al. (1997). The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm. *ACM SIGCOMM Computer Communication Review*.

- RFC 5681 — TCP Congestion Control.

- RFC 3168 — The Addition of Explicit Congestion Notification (ECN) to IP.